# Adaptive Candidate Routing for Distributed Occupant Recognition Across Multiple Building IoT-enabled Smart Spaces.

First Author[1,2*], Second Author[2,3†] and Third Author[1,2†]

[1*]Department, Organization, Street, City, 100190, State, Country.
[2]Department, Organization, Street, City, 10587, State, Country.
[3]Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;
Contributing authors: iiauthor@gmail.com; iiiauthor@gmail.com;
†These authors contributed equally to this work.

**Abstract**

In a distributed state transition system (DSTS) for multi-building camera networks, a face embedding captured in one building must be traced to the building-specific state transition system (BSTS) in which its occupant is registered, before the recognition can be translated into an event and the states updated. A centralized master gallery of face embeddings makes this easy but keeps sensitive biometric data in one place. We propose a routing strategy that avoids it. Each BSTS publishes one Bloom filter built from locality-sensitive hashing (LSH) codes of the face embeddings of its registered occupants. A captured face embedding is converted into $\boldsymbol{L}$ LSH codes that are checked against all filters, and only the best-scoring BSTSs verify it against their own reference face embeddings. On 500 CASIA-WebFace identities in 10 buildings, averaged over 20 random building assignments, the default setting identifies 92.37% of test face embeddings correctly, against 95.47% for a centralized search with the same vote rule; an adaptive rule that asks extra BSTSs only when the ranking is unclear reaches 92.44% and contacts 2.54 BSTSs on average. The mean face embedding and the accept angle come from reference face embeddings only; $\boldsymbol{K}$ and $\boldsymbol{P}$ were chosen after seeing test accuracy, so alternatives are reported. LSH code collisions raise the real false-positive rate of the filters to 3.54% instead of the configured 1%. When the same occupants are split over 2, 5 and 10 BSTSs, the adaptive rule stays within 0.5 points of the centralized search at 2 and 5 BSTSs (95.0% and 95.3% against 95.5%) and 2.9 points below it at 10 (92.6%), and someone who holds the filters can tell which building an occupant is registered in (AUC

0.93). Costs are computed, not measured; encryption and unknown individuals are not evaluated.

**Keywords:** Occupant identification and tracking, Distributed state transition system, Smart Spaces, Locality-sensitive hashing, Bloom filter, Privacy in data analytics

# 1 Introduction

Large camera networks that monitor several buildings can be abstracted as a distributed state transition system (DSTS), composed of independent building-specific state transition systems (BSTSs) in which recognition events drive state transitions [1]. Every occupant is registered in exactly one BSTS, and an occupant seen in another building is a visitor there. When a face embedding is captured in one building, the system must therefore find the BSTS in which its occupant is registered before the recognition can be translated into an event and the states updated. The DSTS framework resolves such visitors against a centralized master list of registered occupants [1], that is, one central gallery with the face embeddings of everyone. This is simple and accurate, but it stores sensitive biometric data in a single place, which is an attractive target and may conflict with data-minimization rules. The other extreme is a gallery in each BSTS and a request to every BSTS for every captured face embedding, which exposes every captured face embedding to every building.

We study the step between these two: a routing step that decides which few BSTSs should be asked. Each BSTS publishes one Bloom filter [2] built from LSH codes of the face embeddings of its registered occupants. A captured face embedding is checked against all filters, and only the best-scoring BSTSs receive it. Reference face embeddings are not published. The idea of publishing a compact summary of what each site holds, and checking the summaries before contacting a site, is used in web cache sharing [3]. Face matching is approximate, so the summary cannot hold exact keys; it holds LSH codes instead.

*Scope.* This paper is about the routing step. We measure how accurate it is, how many BSTSs it contacts, how many false positives the filters give, and how it behaves with 2, 5 and 10 BSTSs. All evaluated face embeddings belong to registered occupants; identifying unknown individuals, who are registered in no BSTS, is left for future work. We do not implement or test any cryptographic protection. We assume that messages between BSTSs use a standard authenticated, encrypted channel, and Section 3.1 states what is and is not protected. Storage and communication costs are computed from message sizes; no distributed run was measured.

Our contributions are:

- a routing design with one Bloom filter of LSH codes per BSTS, and a way to choose the code length $K$ and the number of codes $L$ from measured angle distributions;
- measurements on 500 identities and 20 building assignments: accuracy against a centralized search, the gain from multi-probe hashing, and a real false-positive rate of 3.54% that we trace to collisions between LSH codes;

- a rule that asks extra BSTSs only when the two best scores are close, which is as accurate as asking all shortlisted BSTSs and contacts about half as many;
- an evaluation with 2, 5 and 10 BSTSs, an analytic cost comparison, and a measurement of what the published filters reveal about the building in which an occupant is registered.

## 2 Related Work

Occupant identification and tracking in smart spaces has been modelled with state transition systems whose events abstract the recognition step and whose transition function abstracts the reasoning behind state changes [4]. The DSTS extends this to multi-building camera networks as independent BSTSs, and resolves a visitor against a centralized master list of registered occupants [1]; we replace that lookup by routing. Other occupant tracking work uses RF signals [5], camera-based surveillance analysis such as video synopsis [6], and quantitative metrics for smart spaces [7]. Across multi-building smart spaces, computational efficiency and privacy preservation remain the two main challenges, especially with camera-based sensing [8]. Facial biometrics suit identification in IoT-enabled smart spaces [9] because they carry distinctive identity information and can be captured remotely, without the person stopping, cooperating, or presenting a credential.

*Template protection and Bloom filters.* Rathgeb et al. [10] apply Bloom filters to iris templates. Closest to our work, Marzani et al. [11] turn face features into identifiers with a fuzzy extractor and store them in homomorphically encrypted Bloom filters to count people across locations. Their goal is counting by exact identifier membership; ours is routing a face embedding by approximate similarity (LSH codes with multi-probe) to the BSTS in which its occupant is registered.

*Routing in distributed search.* Summary Cache [3] lets web proxies exchange Bloom filters of their contents and contact only matching proxies; our routing step follows this idea with LSH codes in place of exact keys. For vector data, Gottesbüren et al. [12] study clustering- and LSH-based methods that send a search request only to shards likely to hold its neighbours. Semord [13] places similar vectors close together in a decentralized overlay using a learned hash; its peers exchange summaries that include representative vectors, and it is evaluated on generic benchmarks, not on face embeddings or with a privacy goal. These systems route by vector placement, centroids or learned structure, which requires moving data or sharing vector summaries. We instead let each BSTS publish a small Bloom filter of LSH codes and keep its reference face embeddings. To our knowledge, the combination of LSH codes and per-BSTS Bloom filters has not been measured for this task before.

## 3 Methodology

The method has two phases: each BSTS builds its Bloom filter once (Algorithm 1), and every captured face embedding is then routed, verified, and translated into a recognition event (Algorithm 2). We first state the setting and what is protected, then define the quantities that the algorithms use.

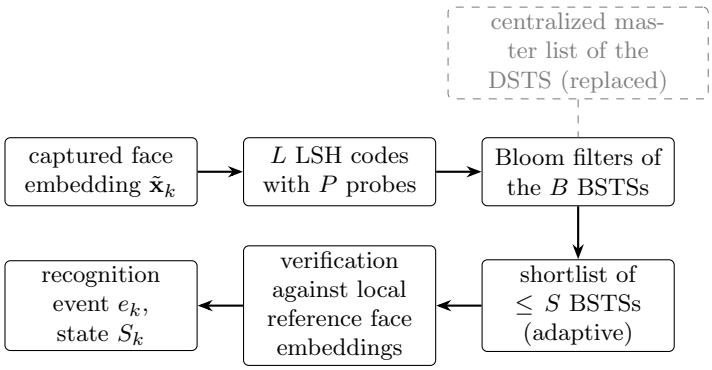



**Fig. 1** Routing in a DSTS. A captured face embedding is checked against the published Bloom filter of every BSTS, only the shortlisted BSTSs verify it with their private reference face embeddings, and the result is translated into a recognition event that updates the state.

## 3.1 System and Threat Model

We build on the distributed state transition system (DSTS) of [1] (Figure 1), which abstracts a multi-building camera network as independent building-specific state transition systems (BSTSs). Each $\text{BSTS}_b$ has a set of states, a set of events and a transition function $\Delta^b : S^b \times E^b \to S^b$; a state records the probabilities of occupants being present in the zones of the building [1, 4]. $\text{BSTS}_b$ stores the reference face embeddings of its $N$ registered occupants; an occupant registered in another BSTS is a visitor there. A face captured at event instance $k$ in a zone of building $q$ is converted into a face embedding. Recognition of this face embedding is translated into a recognition event $e_k$, and the transition function derives the next state $S_k$ from the current state $S_{k-1}$. When the occupant is registered in another BSTS, the DSTS looks the face embedding up in a centralized master list of all registered occupants [1]; this paper replaces that lookup by the routing step below and evaluates the identification step only. We assume that every occupant is registered in exactly one BSTS. Each BSTS constructs a Bloom filter from the Locality-Sensitive Hashing (LSH) codes of the face embeddings of its registered occupants, and the system uses these filters to shortlist candidate BSTSs, which verify the captured face embedding against their stored reference face embeddings. Reference face embeddings are not exchanged between BSTSs during routing or verification, although the captured face embedding is sent to shortlisted BSTSs. We assume that BSTSs execute these operations correctly but may attempt to infer sensitive information from received data. Published Bloom filters, public hash parameters and captured face embeddings may reveal building membership, as evaluated in Section 5.6. Authenticated and encrypted communication is assumed but not implemented or evaluated. Candidate-face guessing, external information, repeated captures, colluding BSTSs, malicious behavior, and traffic analysis are outside the scope of this study.

### 3.2 Face Embedding and LSH Codes

For the face $o_k$ captured at event instance $k$, SCRFD-10GF [14] detects the face and pretrained ArcFace (InsightFace `buffalo_l`) [15] produces a 512-dimensional raw face embedding $\tilde{\mathbf{x}}_k$; reference face embeddings are produced in the same way. We first unit-normalize and then subtract the reference-only population mean $\bar{\mathbf{x}}$ and renormalize, giving the face embedding $\mathbf{p}$ used for routing:

$$\mathbf{x}_k = \frac{\tilde{\mathbf{x}}_k}{\|\tilde{\mathbf{x}}_k\|_2}, \qquad \mathbf{p} = \frac{\mathbf{x}_k - \bar{\mathbf{x}}}{\|\mathbf{x}_k - \bar{\mathbf{x}}\|_2}. \tag{1}$$

The same mean and transformation are used for enrollment and identification. Let $K$ be the number of bits per code and $L$ the number of slots. Using $KL$ fixed-seed Gaussian hyperplanes $\mathbf{h}_{\ell,i}$, define $b_{\ell,i} = \mathbb{1}[\mathbf{p} \cdot \mathbf{h}_{\ell,i} \geq 0]$ and the slot-tagged item

$$c_\ell = \sum_{i=0}^{K-1} b_{\ell,i} 2^i, \qquad s_\ell = \ell 2^K + c_\ell. \tag{2}$$

For angle $\theta$ between two unit face embeddings, a bit matches with probability $p_{\text{bit}} = 1 - \theta/180$ [16].

### 3.3 Parameter Selection

Averaging $p_{\text{bit}}^K$ over reference-only genuine and impostor pairs of face embeddings gives $p_{\text{code}}^{(G)}$ and $p_{\text{code}}^{(I)}$. Assuming independent slots,

$$\text{Recall} = 1 - (1 - p_{\text{code}}^{(G)})^L, \quad \text{FPR}_{\text{imp}} = 1 - (1 - p_{\text{code}}^{(I)})^L. \tag{3}$$

For a recall target $\rho$, set $L = \left\lceil \ln(1-\rho) / \ln(1 - p_{\text{code}}^{(G)}) \right\rceil$. For each candidate $K$ we compute $L$, reject values with predicted $\text{FPR}_{\text{imp}} > \varphi$ for a chosen bound $\varphi$, and keep the pair with the smallest $KL$.

### 3.4 Per-BSTS Bloom Filters

Each BSTS publishes a Bloom filter but retains its reference face embeddings. For registered occupant $u$ in $\text{BSTS}_b$, the normalized mean of the $R$ reference face embeddings forms centroid $\mathbf{z}_{b,u} = \text{Normalize}(R^{-1} \sum_{r=1}^{R} \mathbf{e}_{b,u,r})$, which is encoded into $L$ items using Equation 2. With $n = NL$ inserted items and target false-positive probability $f^\star$, the filter size and number of hash positions are

$$m = \left\lceil \frac{-n \ln f^\star}{(\ln 2)^2} \right\rceil, \qquad \kappa = \text{round}\left( \frac{m}{n} \ln 2 \right). \tag{4}$$

We use Kirsch–Mitzenmacher double hashing [17]: $h_1 = \text{SM64}(s \oplus (2\sigma + 1))$, $h_2 = \text{SM64}(s \oplus (2\sigma + 2)) \vee 1$, and $p_j = ((h_1 + j h_2) \bmod 2^{64}) \bmod m$ for $j = 0, \ldots, \kappa - 1$,

**Algorithm 1** Building the published Bloom filter at $\mathrm{BSTS}_b$

**Input:** Raw reference face embeddings $\tilde{\mathbf{x}}_{b,u,r}$ of the $N$ registered occupants of $\mathrm{BSTS}_b$ ($R$ per occupant);
population mean $\bar{\mathbf{x}}$; shared hyperplanes $\mathcal{H}$ and salt $\sigma$; parameters $K, L, f^\star$.
**Output:** Published Bloom filter $F_b$; private reference face embeddings $\mathcal{E}_b$.

$n \leftarrow NL$; derive $m$ and $\kappa$ ▷ Equation 4
$F_b \leftarrow$ all-zero bit array of length $m$
**for** $u = 1$ to $N$ **do**
  **for** $r = 1$ to $R$ **do**
    $\mathbf{e}_{b,u,r} \leftarrow$ preprocessed $\tilde{\mathbf{x}}_{b,u,r}$ ▷ Equation 1
  **end for**
  $\mathbf{z}_{b,u} \leftarrow \mathrm{Normalize}\big(R^{-1} \sum_r \mathbf{e}_{b,u,r}\big)$ ▷ occupant centroid
  **for** $\ell = 0$ to $L - 1$ **do**
    $s_\ell \leftarrow$ slot-tagged item of $\mathbf{z}_{b,u}$ ▷ Equation 2
    set the $\kappa$ bits $p_0, \ldots, p_{\kappa-1}$ of item $s_\ell$ in $F_b$ ▷ double hashing
  **end for**
**end for**
$\mathcal{E}_b \leftarrow \{\mathbf{e}_{b,u,r}\}$, kept private; publish $F_b$

where $\sigma$ is a shared salt and SM64 is the SplitMix64 finalizer [18]. A check is positive when all $\kappa$ positions are set.

## 3.5 Routing, Verification, and Event Generation

For each slot, multi-probe routing [19] checks the original code and $P$ single-bit variants obtained by flipping the bits with the smallest margins $|\mathbf{p} \cdot \mathbf{h}_{\ell,i}|$:

$$s'_{\ell,j} = \ell 2^K + \big(c_\ell \,\mathrm{XOR}\, 2^{w_{\ell,j}}\big), \quad j = 1, \ldots, P, \tag{5}$$

where $w_{\ell,j}$ is the index of the $j$-th smallest-margin bit. $\mathrm{BSTS}_b$ receives score $A_b$, the number of slots with at least one positive check; the shortlist $\mathcal{C}$ contains the top $\min(S, B)$ BSTSs by score (ties by index). In a shortlisted BSTS, each reference face embedding votes for its registered occupant when its angle to the captured face embedding is at most the accept angle $\tau$, the cutoff that minimizes the sum of the genuine false-reject rate and the impostor false-accept rate on reference face embeddings; the local winner is the registered occupant with most votes (ties by smallest reference angle), and acceptance requires at least $V$ of $R$ votes. The adaptive policy verifies all shortlisted BSTSs when $g = A_{(1)} - A_{(2)} \le g_0$ and selects by vote count, minimum reference angle, then routing rank; otherwise it stops at the first local acceptance in routing order. An accepted occupant is translated into the recognition event $e_k$ of the BSTS in which it is registered, and the transition function updates the state. If none is accepted, the system abstains, no event is generated and the state is left unchanged. The default all-shortlist rule used in the multi-probe comparison instead compares votes across all registered occupants of the shortlisted BSTSs, breaking ties by minimum reference angle.

**Algorithm 2** Adaptive candidate routing, event generation and state generation

**Input:** Face embedding $\tilde{\mathbf{x}}_k$ captured at event instance $k$ in zone $j$ of building $q$;
published filters $\{F_b\}_{b=1}^{B}$; private reference face embeddings $\{\mathcal{E}_b\}_{b=1}^{B}$ held by the BSTSs;
mean $\bar{\mathbf{x}}$, hyperplanes $\mathcal{H}$, parameters $K, L, P, S, V, \tau, g_0$;
current state $S_{k-1}$.

**Output:** Recognised occupant $a^\star$ registered in $\text{BSTS}_{b^\star}$ (or ABSTAIN), event $e_k$ and next state $S_k$.

$\mathbf{p} \leftarrow$ preprocessed $\tilde{\mathbf{x}}_k$ ▷ Equation 1
**for** $\ell = 0$ to $L-1$ **do**
    compute $s_\ell$ and the $P$ probe items $s'_{\ell,1}, \ldots, s'_{\ell,P}$ ▷ Eqs. 2, 5
**end for**
**for** $b = 1$ to $B$ **do**
    check the original and probe items of every slot against $F_b$
    $A_b \leftarrow$ number of slots with at least one positive check ▷ each slot once
**end for**
$\mathcal{C} \leftarrow$ top $\min(S, B)$ BSTSs by decreasing $A_b$ ▷ ties by index
$g \leftarrow A_{(1)} - A_{(2)}$ if $B \geq 2$, else $g \leftarrow +\infty$
$a^\star \leftarrow$ ABSTAIN
**if** $g \leq g_0$ **then** ▷ ranking unclear: verify all shortlisted BSTSs
    **for all** $b \in \mathcal{C}$ **do**
        $(u_b, v_b) \leftarrow \textsc{LocalVerify}(b, \mathbf{p})$
    **end for**
    $\mathcal{A} \leftarrow \{b \in \mathcal{C} : v_b \geq V\}$
    **if** $\mathcal{A} \neq \emptyset$ **then**
        $(a^\star, b^\star) \leftarrow (u_b, b)$ for the $b \in \mathcal{A}$ with most votes ▷ ties: angle, then rank
    **end if**
**else** ▷ ranking clear: sequential first-match
    **for all** $b \in \mathcal{C}$ in routing order **do**
        $(u_b, v_b) \leftarrow \textsc{LocalVerify}(b, \mathbf{p})$
        **if** $v_b \geq V$ **then**
            $(a^\star, b^\star) \leftarrow (u_b, b)$; **break**
        **end if**
    **end for**
**end if**
$e_k \leftarrow \mathit{event_gen}(a^\star, b^\star, k, q, j)$ ▷ recognition translated into an event; none on ABSTAIN
$S_k \leftarrow \mathit{state_gen}(S_{k-1}, e_k)$ ▷ transition function $\Delta$; $S_k = S_{k-1}$ on ABSTAIN
**return** $a^\star, e_k, S_k$
**function** $\textsc{LocalVerify}(b, \mathbf{p})$ ▷ runs in $\text{BSTS}_b$ on its private reference face embeddings
    **for all** registered occupant $u$ of $\text{BSTS}_b$ **do**
        $v_u \leftarrow |\{r : \angle(\mathbf{p}, \mathbf{e}_{b,u,r}) \leq \tau\}|$
    **end for**
    **return** $u^\star = \arg\max_u v_u$ (ties by smallest reference angle) and $v_{u^\star}$
**end function**

# 4 Experimental Setup

We use the face embeddings of 500 identities of CASIA-WebFace [20], registered in $B = 10$ BSTSs of $N = 50$ occupants each. Each occupant has $R = 20$ reference face embeddings and 20 held-out test face embeddings, giving 20,000 face embeddings.

Every one of the 10,000 test face embeddings is processed as one event instance: it is routed over all $B$ BSTSs and its recognition is translated into an event. The evaluation is *closed-set*: every test face embedding belongs to an occupant registered in exactly one BSTS. We use $K = 11$, $L = 122$, $P = 3$, $S = 5$, $V = 12$ and $f^\star = 0.01$. With these settings each filter stores $n = NL = 6{,}100$ items and has $m = 58{,}469$ bits and $\kappa = 7$ hash positions (about 7.3 KB). For parameter selection (Section 3) we use a recall target $\rho = 0.90$, an impostor-FPR bound $\varphi = 0.10$ and $K \in \{4, \ldots, 15\}$; the model selects $(K, L) = (10, 88)$, whereas the main experiments use $(11, 122)$ because it performed better on test accuracy (Section 5.1).

*Use of test data.* The mean face embedding, the accept angle and the angle statistics behind $K$ and $L$ are estimated from reference face embeddings only: the mean is taken over the 10,000 reference face embeddings, genuine pairs are two reference face embeddings of the same occupant, and impostor pairs are reference face embeddings of different occupants. This gives $\tau = 80.43°$. An earlier version used test face embeddings for the mean and $\tau$ (80.54°); with reference-only estimates, correct-identification rates changed by at most 0.06 points, and misidentification rates fell by 0.1 to 0.15 points. Since $K = 11$ and $P = 3$ were chosen after looking at test accuracy, we also report $K = 10$, the model's choice, and the full sweep over $P$ (Section 5.1).

*Seeds.* Each result is repeated for 20 seeds. A seed changes three things together: the assignment of the 500 identities to BSTSs of 50 registered occupants, the LSH hyperplanes, and the Bloom filter salt. The face embeddings, their mean and the accept angle stay fixed, so the centralized baseline is a single number. We report the mean over seeds with a 95% confidence half-width ($\pm$, Student-$t$, 19 degrees of freedom) and, for comparisons, the paired difference over seeds. For $B = 2$ and $B = 5$, we regroup the occupants of the first two and first five buildings of the original layout into BSTSs of 50. A single run with the original assignment and hyperplane seed 42 gave 92.32% at $P = 3$ with the default rule, 0.05 points below the 20-seed mean.

*Metrics.* Each test face embedding results in exactly one of four recognition outcomes: *correct* (the event names the right registered occupant); *misidentified* (a wrong occupant is accepted, the most serious error); *abstained-reject* (the true BSTS was shortlisted but nobody reached $V$ votes); and *abstained-routing* (the true BSTS was not shortlisted and nobody reached $V$ votes). The centralized baseline gives 95.47% correct, 1.41% misidentified and 3.12% abstained.

# 5 Results

## 5.1 Choice of $K$ and $L$

The analytic model uses only the angle arrays, so it has no seed. Using the mean angle first gives $p_{\text{code}}^{(G)} = 0.0115$ at $K = 11$, against the correct array average of 0.0187, which is about 39% too low. The model picks $K = 10$, $L = 88$. In the full pipeline at $P = 3$ with the default rule, $K = 11$, $L = 122$ is more accurate than $K = 10$ by $0.52 \pm 0.13$ points (paired) and better in 20 of 20 seeds: correct 92.37% against 91.85%, mainly because routing failures fall from 4.88% to 4.46%. Both results are reported; we use $K = 11$ in the rest of the paper, and we note that this choice was made after seeing test accuracy.

**Table 1** Multi-probe sweep at $B = 10$ ($K = 11$, $L = 122$, default rule). Results are reported as the mean over 20 seeds ($\pm$ 95% CI), in %. The number of items looked up per filter and face embedding is $L(1 + P)$.

| $P$ | Correct | Misid. | Abst.-reject | Abst.-routing | Items |
|---|---|---|---|---|---|
| 0 | $88.28 \pm 0.13$ | $1.91 \pm 0.04$ | $1.74 \pm 0.03$ | $8.07 \pm 0.11$ | 122 |
| 1 | $90.92 \pm 0.09$ | $1.61 \pm 0.04$ | $1.71 \pm 0.04$ | $5.76 \pm 0.11$ | 244 |
| 2 | $91.90 \pm 0.09$ | $1.45 \pm 0.04$ | $1.73 \pm 0.05$ | $4.92 \pm 0.10$ | 366 |
| 3 | $92.37 \pm 0.08$ | $1.42 \pm 0.03$ | $1.75 \pm 0.04$ | $4.46 \pm 0.08$ | 488 |
| 5 | $92.80 \pm 0.07$ | $1.38 \pm 0.03$ | $1.77 \pm 0.05$ | $4.05 \pm 0.07$ | 732 |
| 7 | $92.89 \pm 0.07$ | $1.38 \pm 0.03$ | $1.76 \pm 0.05$ | $3.97 \pm 0.08$ | 976 |

## 5.2 Multi-Probe and Decentralized versus Centralized Identification

Table 1 varies $P$ at $B = 10$ with the default rule. Without probing, $8.07 \pm 0.11\%$ of face embeddings are lost to routing misses. Multi-probe with $P = 3$ reduces this to $4.46 \pm 0.08\%$ (a 45% relative reduction), raises correct identification from 88.28% to 92.37% (a paired gain of $4.09 \pm 0.15$ points, positive in 20 of 20 seeds) and lowers misidentification from 1.91% to 1.42%. The gains get smaller as $P$ grows, while the number of lookups grows linearly, so we use $P = 3$ as a compromise on cost, not as the best possible value.

We now compare the decentralized system with the centralized baseline. At $B = 2$ and $B = 5$ the shortlist of $S = 5$ covers every BSTS, so routing cannot fail and the two systems give the same result in every seed (95.75% correct at $B = 2$ and 95.64% at $B = 5$). At $B = 10$ the decentralized system is $3.10 \pm 0.08$ points behind the baseline, and 4.46% of face embeddings are routing failures. Before multi-probe the gap was 7.19 points, so multi-probe closes 57% of it. Misidentification (1.42% against 1.41%) hardly changes, so the loss comes from missed matches and not from more wrong accepts.

## 5.3 Effective False-Positive Rate

The 1% target describes the bit array for items that are independent of the stored ones. We measured the rate the system really has by building all ten filters and testing the enrolled items of the *other nine* BSTSs against each one. The real per-item rate is $3.54 \pm 0.02\%$ over the 20 seeds. When we compare the integer codes directly, without a Bloom array, $2.61 \pm 0.02\%$ of the items of other BSTSs already equal an enrolled item in the same slot. This is close to the 2.41% expected if the codes of 50 registered occupants were evenly spread over $2^{11} = 2048$ values, namely $1 - (1 - 2^{-K})^N$. The Bloom array alone, tested on random integers that were never inserted, gives $0.94 \pm 0.01\%$ at 51.3% fill, close to its design rate. The extra false positives therefore come from the LSH codes, not from the Bloom array.

A larger $K$ gives more possible codes and lowers the real false-positive rate, but $L$ must grow to keep recall at 0.90. At $K = 13$, $L = 230$ each filter grows from 7.3 KB to about 13.8 KB and the real rate falls to 1.67%. With $P = 6$ it is $0.21 \pm 0.08$ points more accurate than the best $K = 11$ setting ($P = 7$, 92.89%; paired, better in 19 of 20

seeds) and $0.73 \pm 0.11$ points more accurate than the default (93.10% against 92.37%; better in 20 of 20 seeds), but it needs 1,610 lookups per filter and face embedding against 488 or 976.

## 5.4 Verification Policies

Figure 2(a) shows the policies of Section 3.5 and the fixed shortlist sizes; the figures in this paragraph are the 10,000 test face embeddings of the main experiment. These results use the policies' own rule for choosing among BSTSs (best registered occupant per BSTS, ties by rank order), so verify-all gives 92.34% here and not the 92.37% of the default rule in Table 1. The two differ only when two shortlisted BSTSs return the same number of votes, which happens for 72.5 of 10,000 face embeddings on average. Each captured face embedding is scored against all ten filters, including that of the occupant's home BSTS, so there is no separate local-recognition stage. Sequential first-match raises the misidentification rate to 2.17%, because a wrong BSTS ranked above the true one sometimes confirms first. Asking all five BSTSs lowers it to 1.45% and gains $0.71 \pm 0.03$ points, but it contacts 5 BSTSs per face embedding. The adaptive rule with $g_0 = 5$ is slightly more accurate than asking all five ($+0.10 \pm 0.02$ points, paired, better in 20 of 20 seeds) while contacting 2.54 BSTSs on average; 37.6% of the face embeddings ask all five. All three policies abstain on exactly the same face embeddings, because acceptance depends only on whether some shortlisted BSTS reaches $V$ votes.

## 5.5 Cost

Costs are computed from message sizes and the measured number of BSTSs asked; they are not measured in a distributed run, because all calls were in-process. A face embedding has 512 single-precision values (2,048 bytes). Routing replaces a 20.5 MB master gallery in one place with ten published filters of 7.3 KB each (the BSTSs keep 2.05 MB of private reference face embeddings each). Handling one captured face embedding costs 3.1 KB with sequential asking (1.50 BSTSs), 5.2 KB with the adaptive rule (2.54 BSTSs) and 10.2 KB when all five shortlisted BSTSs are asked, against 20.5 KB for all ten BSTSs and 2.0 KB for a master gallery. Each face embedding still needs $B \cdot L(1 + P) = 4{,}880$ filter lookups, which grow linearly with $B$.

## 5.6 What the Filters Reveal

We measured the simplest attack. Someone who holds all published filters and the public hash parameters scores a face embedding against every filter, in the same way as routing does. With $P = 3$, this score separates the registered occupants of a BSTS from everyone else with an AUC of 0.93 (0.88 without probing), averaged over BSTSs and 10 seeds, and the highest-scoring BSTS is the occupant's own for 79.7% of the face embeddings (66.7% without probing). The filters therefore leak which building an occupant is registered in. This is not a way to recover a face, but it is a real leak, and it grows with the accuracy of the routing itself. We did not test attacks that use many face embeddings, candidate guessing or outside information, so we make no claim of biometric privacy.

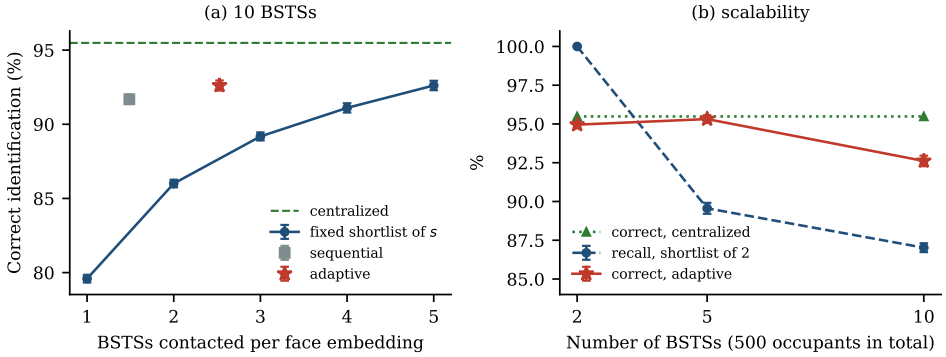



**Fig. 2** (a) Correct identification against the number of BSTSs contacted at $B = 10$: fixed shortlist of $s$ BSTSs, sequential first-match, the adaptive rule and the centralized search. (b) Recall of a shortlist of two and correct identification (adaptive rule and centralized search) for 2, 5 and 10 BSTSs. Mean over 10 seeds with 95% intervals; 2,000 test face embeddings per seed.

### 5.7 Fewer and More BSTSs

The corpus has 500 identities, so we split the same 500 registered occupants over $B = 2$, 5 and 10 BSTSs of 250, 100 and 50 occupants. Each filter is sized for its own number of occupants. We use 2,000 test face embeddings per seed (4 per occupant) and 10 seeds; this subset gives 92.6% for the adaptive rule at $B = 10$, against 92.44% on all 10,000 face embeddings. The shortlist holds at most $S = \min(5, B)$ BSTSs, so at $B = 5$ it covers every BSTS (recall 100%), and Figure 2(b) therefore reports recall for a shortlist of two. That recall falls from 100% at $B = 2$ to 89.6% at $B = 5$ and 87.0% at $B = 10$, because the shortlist is a smaller share of the BSTSs. Correct identification with the adaptive rule is 95.0% and 95.3% at $B = 2$ and 5, close to the 95.5% of the centralized search, and 92.6% at $B = 10$, where the adaptive rule contacts 2.5 BSTSs. The loss at $B = 10$ comes from choosing among more BSTSs, not from more false hits per filter.

## 6 Limitations and Conclusion

*Limitations.* All results use the same 500 identities from one public dataset; the intervals reflect building assignments and hash randomness, not a different population. The values $K = 11$ and $P = 3$ were chosen after seeing test accuracy, and we report alternatives next to them. CASIA-WebFace data come from the web, not from deployed camera networks, so real accuracy is probably lower, and we did not check how much the training data of the pretrained ArcFace model overlaps with CASIA-WebFace. The largest network has 10 BSTSs of 50 registered occupants. The evaluation is closed-set; a first test with held-out identities showed that the vote rule ($\tau$ and $V = 12$) can accept a noticeable share of them, so it must be tightened before unknown individuals are handled. We evaluated the identification step only, not the generated events, the mapping of recognition scores to occupant-presence probabilities, or the state updates

over a sequence of events. Costs are computed from message sizes, not measured in a distributed run. We did not implement or evaluate the encrypted channel, visit-length template storage, or attacks beyond the membership test of Section 5.6. A Bloom filter cannot delete items, so a change in registration needs a rebuild.

*Conclusion.* Per-BSTS Bloom filters of LSH codes can route captured face embeddings to the BSTS of the registered occupant without a centralized master gallery. With multi-probe hashing the default setting identifies 92.37% of test face embeddings correctly at 10 buildings against 95.47% for a centralized search, and the adaptive rule reaches 92.44% while contacting about 2.54 BSTSs. The cost is lost recall that grows with the number of BSTSs, and a false-positive rate that LSH collisions raise well above the Bloom filter's design value. The privacy conclusions are limited to what was evaluated: the design avoids publishing reference face embeddings and a master gallery, and it leaks which building an occupant is registered in (AUC 0.93); encryption, stronger attacks and template storage during visits were not evaluated. The next steps are a larger and more varied population, the identification of unknown individuals with a stricter acceptance rule, and measurements of event generation and state updates over event sequences in a distributed deployment.

**Acknowledgement.** The authors gratefully acknowledge Sri Mata Amritanandamayi Devi (Amma), Chancellor, Amrita Vishwa Vidyapeetham, for her inspiration and for providing financial support for the Article Processing Charges (APC) of this publication.

## References

[1] Mohan, L., Menon, V.: Modelling large scale camera networks for identification and tracking: An abstract framework. IET Computer Vision **14**(7), 426–433 (2020)

[2] Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. Communications of the ACM **13**(7), 422–426 (1970)

[3] Fan, L., Cao, P., Almeida, J., Broder, A.Z.: Summary cache: A scalable wide-area web cache sharing protocol. IEEE/ACM Transactions on Networking **8**(3), 281–293 (2000)

[4] Menon, V., Jayaraman, B., Govindaraju, V.: The three Rs of cyberphysical spaces. Computer **44**(9), 73–79 (2011)

[5] Wu, Z., Zhang, D., Shang, Z., Yuan, Y., Gong, H., Wang, B., Lu, Z., Li, Y., Hu, Y., Sun, Q., Chen, Y.: Learning-based tracking-before-detect for unconstrained indoor human tracking using RF signal. IEEE Transactions on Mobile Computing **24**(6), 4777–4793 (2025) https://doi.org/10.1109/TMC.2025.3529501

[6] Namitha, K., Geetha, M., Athi, N.: An improved interaction estimation and optimization method for surveillance video synopsis. IEEE MultiMedia **30**(3), 25–36 (2022)

[7] Mohan, L., Menon, V., Jayaraman, B.: Quantitative metrics for smart spaces. In: 2025 IEEE International Conference on Smart Computing (SMARTCOMP), pp. 450–455 (2025). https://doi.org/10.1109/SMARTCOMP65954.2025.00104

[8] Kim, J., Kim, Y.M., Zahreddine, R., Welge, W.A., Krishnan, G., Ma, S., Wang, J.: Privacy-preserving visual localization with event cameras. IEEE Transactions on Image Processing **34**, 6215–6230 (2025) https://doi.org/10.1109/TIP.2025.3607640

[9] Khan, I., Zedadra, O., Guerrieri, A., Spezzano, G.: Occupancy prediction in IoT-enabled smart buildings: Technologies, methods, and future directions. Sensors **24**(11), 3276 (2024)

[10] Rathgeb, C., Breitinger, F., Busch, C., Baier, H.: On application of Bloom filters to iris biometrics. IET Biometrics **3**(4), 207–218 (2014)

[11] Marzani, F., Ede, T., Heijenk, G., Steen, M.: Head Count: Privacy-Preserving Face-Based Crowd Monitoring. arXiv:2604.14250 (2026)

[12] Gottesbüren, L., Dhulipala, L., Jayaram, R., Lacki, J.: Unleashing Graph Partitioning for Large-Scale Nearest Neighbor Search. arXiv:2403.01797 (2024)

[13] Wang, S., Liu, Y., Hua, Y., Zhang, X., Qian, C.: Semord: Learned Semantic-Preserving Placement and Low-Fanout Routing for Distributed Vector Search. arXiv:2609.25514 (2026)

[14] Guo, J., Deng, J., Lattas, A., Zafeiriou, S.: Sample and computation redistribution for efficient face detection. In: International Conference on Learning Representations (2022)

[15] Deng, J., Guo, J., Xue, N., Zafeiriou, S.: ArcFace: Additive angular margin loss for deep face recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4690–4699. IEEE, Long Beach, CA, USA (2019)

[16] Charikar, M.S.: Similarity estimation techniques from rounding algorithms. In: Proceedings of the 34th Annual ACM Symposium on Theory of Computing, pp. 380–388. ACM, Montreal, QC, Canada (2002)

[17] Kirsch, A., Mitzenmacher, M.: Less hashing, same performance: Building a better Bloom filter. In: Azar, Y., Erlebach, T. (eds.) Algorithms – ESA 2006. Lecture Notes in Computer Science, vol. 4168, pp. 456–467. Springer, Heidelberg (2006)

[18] Steele, G.L., Lea, D., Flood, C.H.: Fast splittable pseudorandom number generators. In: Proceedings of OOPSLA 2014, pp. 453–472. ACM, Portland, OR, USA (2014)

[19] Lv, Q., Josephson, W., Wang, Z., Charikar, M., Li, K.: Multi-probe LSH: Efficient indexing for high-dimensional similarity search. In: Proceedings of the 33rd International Conference on Very Large Data Bases, pp. 950–961. VLDB Endowment, Vienna, Austria (2007)

[20] Yi, D., Lei, Z., Liao, S., Li, S.Z.: Learning Face Representation from Scratch. arXiv:1411.7923 (2014)